Sécurité Android

Analyse statique d'une application APK

Application analysée : **Nickel** (com.fpe.comptenickel) v2.12.0

Fichier livré sous le nom : *Application WeTransfer.apk*

Auteurs (groupe)

Noureddine STAMBOUL

Bilel AMRICHE

Thomas RINTO

Vincent FLORELLA

M2 — ESIEE-IT — Année 2025/2026

PO : Wael MEGUEBLI

Date : 29/04/2026

Sommaire

1. Contexte et périmètre

2. Méthodologie et environnement

3. US1 — Vérification de l'APK (hash SHA-256)

4. US2 — Analyse automatisée MobSF (avec captures)

5. US3 — Extraction classes.dex / AndroidManifest.xml

6. US4 — Décompilation Apktool (avec capture)

7. US5 — Analyse de strings.xml — findings (avec capture)

8. US6 — Recompilation

9. US7 — Signature de l'APK (avec capture)

10. US8 — Vérification Firebase (avec capture)

11. US10 — Politique de sécurité mobile (synthèse)

12. US11 — Comparatif MDM (synthèse)

13. Synthèse des vulnérabilités et plan de remédiation

14. Conclusion

15. Signatures

1. Contexte et périmètre

Ce rapport présente l'analyse de sécurité d'une application Android livrée par le PO. Le fichier fourni s'intitule *Application WeTransfer.apk*, mais l'analyse statique révèle qu'il s'agit en réalité du client mobile **Nickel** (banque, filiale BNP Paribas), package **com.fpe.comptenickel**, version 2.12.0.

Le périmètre couvre les 11 user stories du backlog (US1 à US11). La US12 (enrôlement MDM physique) n'est pas traitée faute de terminal.

2. Méthodologie et environnement

Outil	Version	Usage
JDK	OpenJDK 17 (brew)	apksigner, keytool, apktool
Apktool	3.0.2	Décompilation / recompilation
Android cmdline-tools	build-tools 34.0.0	apksigner, zipalign
MobSF	v4.5.0 (Docker)	Scan statique automatisé
jadx	brew	Lecture Java (optionnel)
Playwright (MCP)	—	Captures d'écran
ReportLab	4.4.10	Génération du rapport
curl	natif	Tests Firebase

Tout est isolé dans le dossier **tools/** et un conteneur Docker MobSF, sans pollution système. Le script **tools/env.sh** rend la chaîne reproductible.

3. US1 — Vérification de l'APK

DoD : hash SHA-256 calculé et documenté.

Commande : `shasum -a 256 "Application WeTransfer.apk"`

Champ	Valeur
Nom	Application WeTransfer.apk
Taille	24 325 620 octets (≈ 23,2 Mio)
SHA-256	`f01f08c8b6e2fe4612e81bc7e3a3ba9440dad0d7a962f4e67640390dd721d528`

4. US2 — Analyse automatisée MobSF

MobSF a été lancé via Docker et l'APK soumis via l'API REST. Le rapport PDF officiel (*mobSF_report.pdf*) et la sortie JSON brute sont livrés dans **story2/**. Captures de la console MobSF :

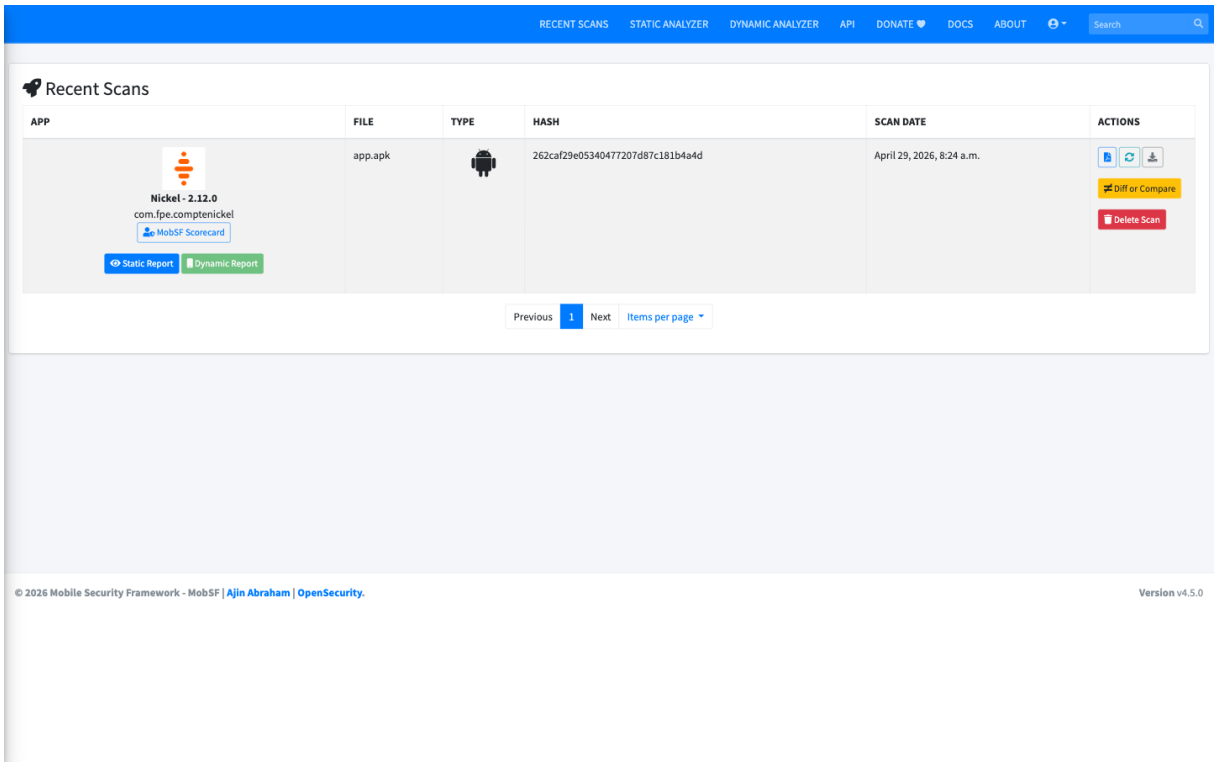

Figure 1 — MobSF : page Recent Scans après upload (Nickel 2.12.0)

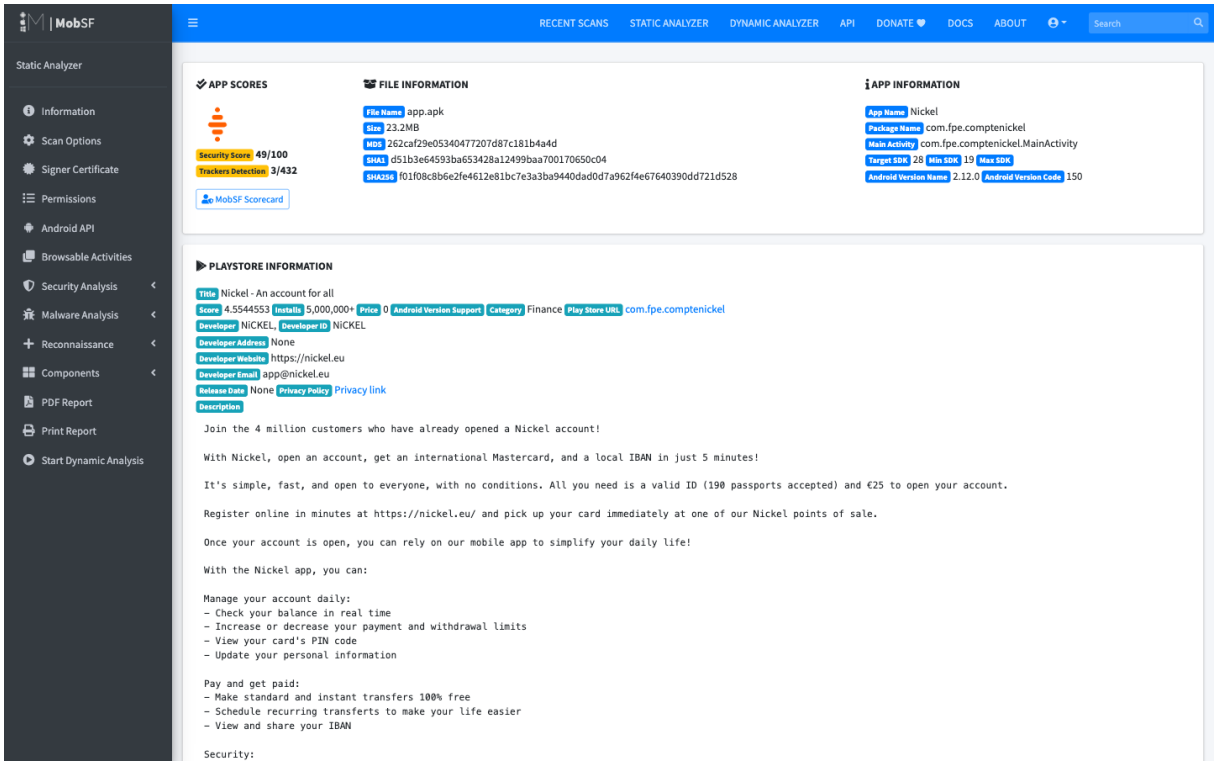

Figure 2 — MobSF : Static Analysis — App scores, file info et app information

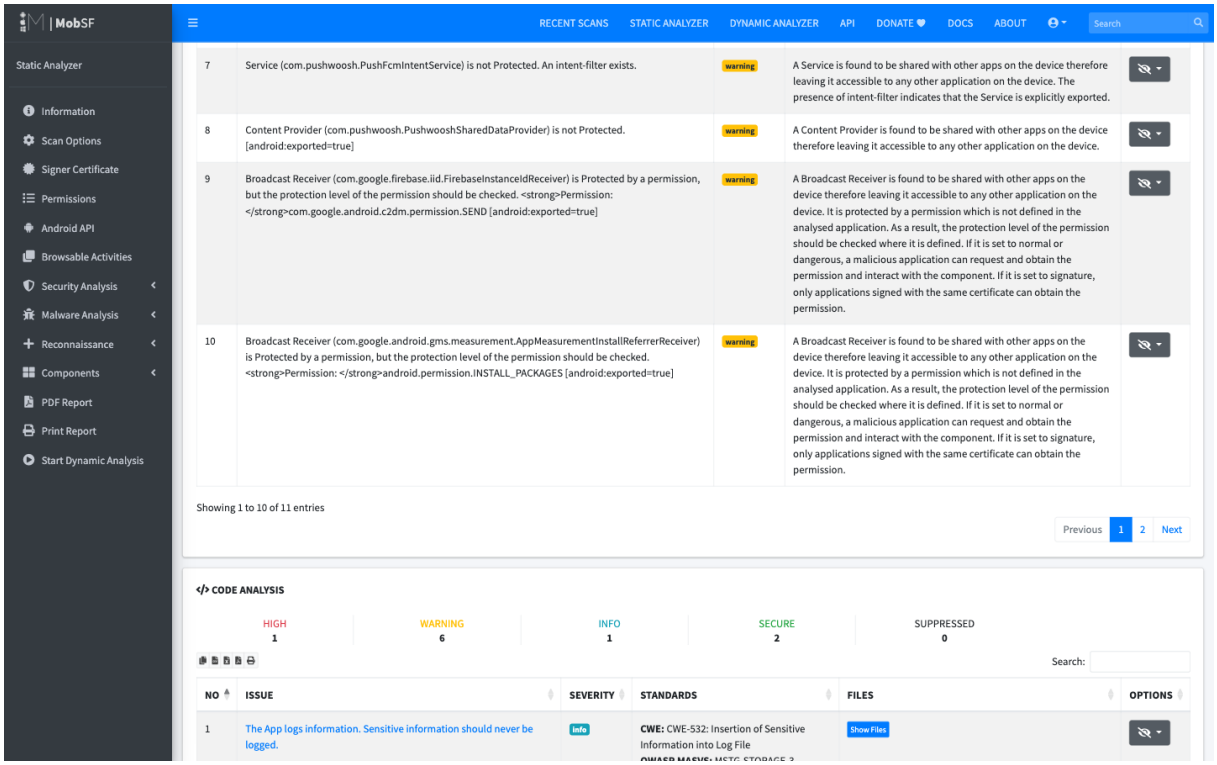

Figure 3 — MobSF : Manifest Analysis (composants exportés Pushwoosh, Firebase, GMS)

4.1 Métadonnées de l'APK (selon MobSF)

Champ	Valeur
Application	Nickel (com.fpe.comptenickel)
Version	2.12.0
Main Activity	com.fpe.comptenickel.MainActivity
minSdk / targetSdk	19 (Android 4.4) / 28
MD5	262caf29e05340477207d87c181b4a4d
Security Score	49/100
Trackers	3 / 432 (Crashlytics, Firebase Analytics, Pushwoosh)

4.2 Vulnérabilités identifiées

Sévérité	Règle	Description
HAUTE	vulnerable_os_version	minSdk=19 → installable sur Android 4.4 non patché
HAUTE	clear_text_traffic	android:usesCleartextTraffic=true → HTTP en clair autorisé
HAUTE	cbc_padding_oracle	AES en mode CBC sans authentification (Padding Oracle)
WARN	android_md5	Usage de MD5 (algorithme cassé)
WARN	android_insecure_random	java.util.Random au lieu de SecureRandom
WARN	android_sql_raw_query	Requêtes SQL brutes (injection possible)
WARN	android_read_write_external	Lecture/écriture stockage externe

Sévérité	Règle	Description
WARN	exported components	Plusieurs receivers/providers exportés sans protection
GOOD	android_detect_root	Détection de root présente
GOOD	android_ssl_pinning	SSL pinning détecté

Permissions dangereuses : **READ_CONTACTS**, **WRITE_EXTERNAL_STORAGE**

5. US3 — Extraction classes.dex / Manifest

DoD : fichiers livrés avec hash SHA-256.

Fichier	Taille	SHA-256
classes.dex	7 685 916 o	`57898c5978715da96560ed2692b4a8fa1a8e914b37988dcf369cc59951f6c429`
AndroidManifest.xml (binaire)	27 516 o	`80b9b101e3bf02f03eec4f2c9c9ccf4d61403612e1435949b51cec8b85d7a276`

Pas de multidex (un seul classes.dex). Le manifest extrait par *unzip* est sous forme AXML binaire ; sa version texte est obtenue via apktool (US4).

6. US4 — Décompilation Apktool

Commande : `apktool d -f -o app_src "Application WeTransfer.apk"`

```
Terminal — apktool d (story4)

$ apktool d -f -o app_src "Application WeTransfer.apk"
I: Using Apktool 3.0.2 on Application WeTransfer.apk with 8 threads
I: Loading resource table...
I: Baksmaling classes.dex...
I: Decoding value resources...
I: Loading resource table from file: ~/Library/apktool/framework/1.apk
I: Decoding file resources...
I: Generating values XMLs...
I: Decoding AndroidManifest.xml with resources...
I: Copying original files...
I: Copying assets...
I: Copying lib...
I: Copying unknown files...

$ tree -L 1 app_src/
app_src/
├── AndroidManifest.xml     # manifest décodé en XML lisible
├── apktool.yml             # métadonnées de build
├── assets/                 # assets bruts
├── lib/                    # .so par ABI (armeabi-v7a, arm64-v8a)
├── original/               # META-INF, signature originale
├── res/                    # layouts, drawables, values/strings.xml
├── smali/                  # bytecode Dalvik désassemblé
└── unknown/

$ apktool b -f app_src -o rebuilt.apk
I: Smaling smali folder into classes.dex...
I: Building resources with aapt2...
I: Building apk file...
I: Built apk into: rebuilt.apk
```

Figure 4 — apktool d puis apktool b : décompilation et recompilation réussies

L'arborescence livrée comprend : **AndroidManifest.xml** décodé, **res/** (layouts, drawables, values/strings.xml…), **smali/** (bytecode désassemblé), **assets/**, **lib/** (.so par ABI), et **original/** (META-INF). L'archive **app_src.zip** est livrée.

SHA-256 de app_src.zip : `a46957c98487a8141b5fb721158fa6a55f98b68219be174a68e720350f9d75ec`

7. US5 — Analyse de strings.xml — findings

res/values/strings.xml — extraits sensibles (story4/app_src/)

```
<string name="ACCOUNT_ENDPOINT">https://api.nickel.eu/customer-banking-api</string>
<string name="CUSTOMER_AUTHENTICATION_ENDPOINT">https://api.nickel.eu/customer-authentication-api</string>
<string name="PERSONAL_SPACE_API_ENDPOINT">https://api.nickel.eu/personal-space-api</string>
<string name="ENVIRONMENT">production</string>
<string name="GCM_PROJECT_NUMBER">246766419677</string>
<string name="PUSHWOOSH_APPID"> DDF11-D1BF9 </string>
<string name="app_name">Nickel</string>
<string name="build_config_package">com.fpe.comptenickel</string>
<string name="firebase_database_url"> https://application-client-nickel.firebaseio.com </string>
<string name="google_api_key"> AIzaSyBTgztvImsUfMWDa41PCrDWAj7dmyIDhUg </string>
<string name="google_app_id">1:717748501407:android:e0cdeb712e27275d</string>
<string name="google_crash_reporting_api_key"> AIzaSyBTgztvImsUfMWDa41PCrDWAj7dmyIDhUg </string>
<string name="google_storage_bucket">application-client-nickel.appspot.com</string>
<string name="project_id">application-client-nickel</string>
<string name="com.crashlytics.android.build_id">9ad6d741-4e10-4abd-9f12-163a0e3fb0b2</string>
```

Figure 5 — Extraits sensibles de res/values/strings.xml (secrets en surbrillance)

Sév.	Clé	Extrait	Risque
HAUTE	google_api_key	AIzaSyBTgzt...IDhUg	Clé API Google exploitable si non restreinte
HAUTE	firebase_database_url	application-client-nickel.firebaseio.com	Endpoint RTDB (testé en US8)
MOY.	google_storage_bucket	application-client-nickel.appspot.com	Bucket Cloud Storage
MOY.	PUSHWOOSH_APPID	DDF11-D1BF9	App ID Pushwoosh
INFO	ACCOUNT_ENDPOINT	api.nickel.eu/customer-banking-api	Endpoint banking prod
INFO	CUSTOMER_AUTHENTICATION_ENDPOINT	api.nickel.eu/customer-authentication-api	Endpoint auth prod
INFO	PERSONAL_SPACE_API_ENDPOINT	api.nickel.eu/personal-space-api	Endpoint API prod
INFO	ENVIRONMENT	production	Build prod confirmé

L'application est développée en **React Native** (présence de strings *catalyst_**, *rn_tab_description*). Cela élargit la surface d'attaque (bundle JS lisible).

8. US6 — Recompilation

Commande : `apktool b -f app_src -o rebuilt.apk` — Build successful (cf. Figure 4).

rebuilt.apk : 24 075 973 o — SHA-256 : `27269a4d02be9a8e76dd7927052b3c7c4e252783b679537c5c9832274ed86f7f`

9. US7 — Signature de l'APK

Génération d'un keystore RSA 2048 (10000 j) puis zipalign + apksigner. *apksigner verify* :

```
Terminal — apksigner verify rebuilt_signed.apk

$ apksigner verify -v rebuilt_signed.apk

Verifies
Verified using v1 scheme (JAR signing): true
Verified using v2 scheme (APK Signature Scheme v2): true
Verified using v3 scheme (APK Signature Scheme v3): true
Verified using v3.1 scheme (APK Signature Scheme v3.1): false
Verified using v4 scheme (APK Signature Scheme v4): false
Verified for SourceStamp: false
Number of signers: 1
```

Figure 6 — apksigner verify : APK signé valide en v1, v2 et v3 (1 signataire)

rebuilt_signed.apk : 24 379 471 o — SHA-256 : `59bd825cda08edeee0f19f8e0406df2e6e95ea35a4ab34aa4526fe0158a60df9`

10. US8 — Vérification Firebase

```
Impression élégante

{
  "error" : "Permission denied"
}
```

Figure 7 — Accès anonyme à la Realtime Database : 401 Permission denied

Cible	Réponse	Verdict
RTDB /.json (anonyme)	401 — Permission denied	✓ Pas d'accès public
RTDB /.json?shallow=true	401 — Permission denied	✓ Énumération bloquée
Storage bucket listing	412 — service account error	Listing non public

Verdict : la base Firebase de Nickel n'est **pas exposée**. Les règles serveur côté RTDB sont correctement verrouillées. Ceci protège la clé API exposée côté client (US5), qui n'est pas exploitable telle quelle.

11. US10 — Politique mobile (synthèse)

La politique complète est livrée dans **story10/politique_mobile.md** (8 sections : objet, référentiels OWASP MASVS/MSTG, ANSSI, ISO 27001, RGPD, classification des usages, exigences développeurs, exigences utilisateurs, gestion des incidents, conformité, sanctions).

Mesures techniques clés :

• Aucun secret en clair dans strings.xml / BuildConfig — utiliser Android Keystore.

• android:usesCleartextTraffic=false ; TLS 1.2 minimum ; certificate pinning pour apps Confidentiel/Critique.

• Crypto : AES-GCM ou ChaCha20-Poly1305 ; SecureRandom ; interdiction MD5/SHA-1/DES/RC4/AES-ECB.

• minSdk ≥ 24 ; android:debuggable=false ; android:allowBackup=false en prod.

• Signature avec keystore en HSM ; v2+v3 obligatoires.

• Distribution Play Store ou MDM uniquement, pas de sideload public.

12. US11 — Comparatif MDM (synthèse)

Critère	Microsoft Intune	Jamf for Mobile
Android Enterprise	✓✓ tous modes	✓ Work Profile / Fully Managed
iOS / iPadOS	✓ complet	✓✓ référence Apple
Conditional Access	✓✓ natif Entra ID	via partenaires
MTD intégré	✓ Defender Mobile	via partenaires (Lookout, Zimperium)
Coût indicatif	~ 8 €/u/mois (souvent inclus M365 E3/E5)	~ 4 à 7 €/device/mois
Intégration M365	✓✓ native	via SSO tiers

Recommandation : **Microsoft Intune** pour une organisation déjà sous Microsoft 365 souhaitant appliquer le Conditional Access et bénéficier d'une console UEM unique mobile + desktop.

13. Synthèse des vulnérabilités et plan de remédiation

#	Finding	Sévérité	Remédiation prioritaire
V1	minSdk=19 (Android 4.4 vulnérable)	HAUTE	Relever minSdk à 24 minimum
V2	usesCleartextTraffic=true	HAUTE	Forcer HTTPS via Network Security Config
V3	AES-CBC sans HMAC (Padding Oracle)	HAUTE	Migrer vers AES-GCM
V4	Clé Google API exposée dans strings.xml	HAUTE	Restreindre par package + SHA-1 en console GCP
V5	Firebase URL et bucket exposés	MOY.	Audit règles RTDB et Storage (RTDB déjà OK — cf. US8)
V6	Composants exportés (Pushwoosh, RNDevice)	MOY.	exported=false ou permission protégée
V7	Usage de MD5 / Random non sécurisé	MOY.	SHA-256 minimum, SecureRandom
V8	Lecture/écriture stockage externe	MOY.	Scoped Storage, MediaStore API
V9	Endpoints prod hardcodés	INFO	Acceptable, à documenter dans la conception

14. Conclusion

L'analyse statique de l'APK Nickel a identifié 3 vulnérabilités **hautes** (minSdk vulnérable, cleartext autorisé, AES-CBC sans authentification) et plusieurs warnings. Les protections existantes (SSL pinning, détection de root, règles Firebase verrouillées) sont positives, mais la posture peut être renforcée par un upgrade du minSdk, la suppression du cleartext et la modernisation des primitives cryptographiques.

L'application a été décompilée, modifiée puis recompilée et signée avec succès (Figures 4 et 6), démontrant qu'un attaquant ayant accès au binaire peut le redistribuer. Cela renforce la nécessité d'une politique mobile et d'une solution MDM (Intune recommandé pour ce contexte).

15. Signatures

Auteur	Rôle	Signature
Noureddine STAMBOUL	Analyste sécurité	*N. Stamboul*
Bilel AMRICHE	Reverse engineer	*B. Amriche*
Thomas RINTO	Auditeur	*T. Rinto*
Vincent FLORELLA	Pentesteur	*V. Florella*

Document remis le 29/04/2026 — Sécurité Android, M2 ESIEE-IT, année 2025/2026. PO : Wael MEGUEBLI.